

NetSDK Programming Manual

(Active Registration)



V1.0.0

Foreword

Purpose

Welcome to use NetSDK (hereinafter referred to be "SDK") programming manual (hereinafter referred to be "the Manual").

SDK, also known as network device SDK, is a development kit for developer to develop the interfaces for network communication among surveillance products such as Network Video Recorder (NVR), Network Video Server (NVS), IP camera (IPC), Speed Dome (SD), and intelligence devices.

This document describes the demo and basic development process for active registration. For details on the basic core service processes including initialization, login, general alarms and intelligent alarms, refer to the *NetSDK Programming Guide*.

The example codes provided in the Manual are only for demonstrating the procedure and not assured to copy for use.

Readers

Software development engineers, project managers, and product managers who use the SDK.

Safety Instructions

The following symbols might appear in the manual.

Symbol	Description
 Tips	Provides methods to help you solve a problem or save time.
 NOTE	Provides additional information as a supplement to the text.

Revision History

Version	Revision Content	Release Time
V1.0.0	First release	May 2026

About the Manual

- This manual is provided for general reference only. Discrepancies might exist between the product and this manual, including but not limited to printed, electronic, or any other editions.
- The manual will be updated according to the latest laws and regulations of related jurisdictions. Specifications and documentation are subject to change without prior notice. For the latest versions, as well as legal notices, consult the printed manual, the product webpage, our official website, or scan the QR code to access the electronic edition.
- The manual will be updated as necessary to reflect product changes and applicable legal requirements. These updates will not diminish your statutory rights, nor will they remove or weaken the safety information or essential instructions provided at the time of purchase.
- Although we make every effort to ensure accuracy, errors or deviations in descriptions of functions, operations, or technical data might occur. If there is any uncertainty or controversy, we reserve the right of final explanation.
- All trademarks, registered trademarks and company names referenced in this manual are the property of their respective owners.

Table of Contents

Foreword	I
Chapter 1 Overview	1
1.1 Introduction.....	1
1.2 Active registration demonstration.....	1
Chapter 2 Actively Registration Demo Operation	2
Chapter 3 Call Active Registration API	3
3.1 Start/stop active registration service.....	3
3.1.1 Introduction	3
3.1.2 Interface overview	3
3.1.3 Process.....	3
3.1.4 Example code	4
Appendix 1 Security Recommendation.....	8

Chapter 1 Overview

1.1 Introduction

NetSDK active registration is mainly used to solve network restriction problems such as public network servers being unable to search for internal network devices, or situations where the device's IP address is not fixed (for example, the device's IP address may be not fixed when it is on a 4G network or Wi-Fi network). It also helps the monitoring server to efficiently configure the equipment, enabling the rapid implementation or recovery of application scenarios.

The active registration functionality involved in NetSDK mainly includes two parts:

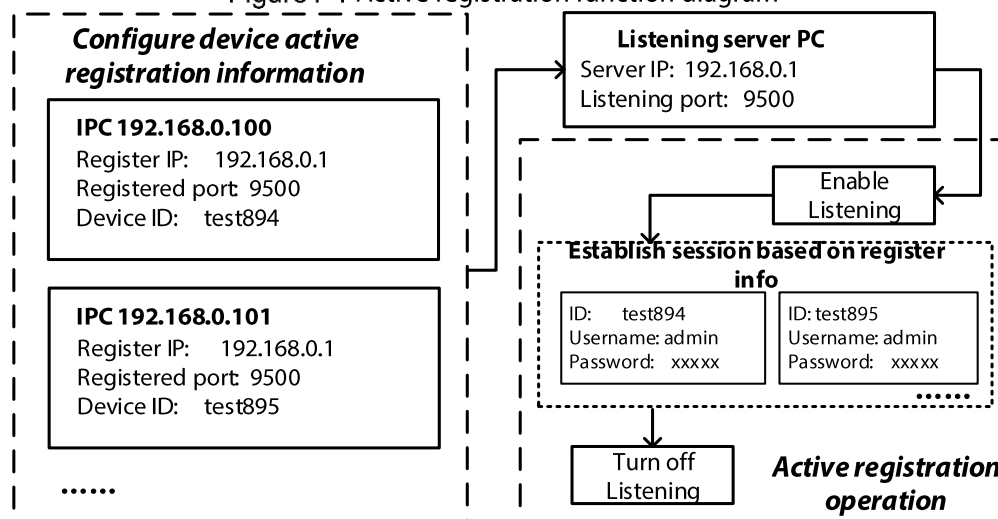
- The target device active registration information is configured via the device web interface.
- Monitor the server startup and shutdown.

The demo provides a basic implementation of the two main functions mentioned above.

1.2 Active registration demonstration

The diagram illustrating the active registration function is shown below.

Figure1-1 Active registration function diagram



Chapter 2 Actively Registration Demo Operation

Configure the device to actively register with the platform. It supports configuring active registration information directly through the device's web interface, and allows for quick addition and deletion of devices that are monitoring to this server, in conjunction with the active registration demo monitoring server.

Chapter 3 Call Active Registration API

3.1 Start/stop active registration service

3.1.1 Introduction

The SDK provides an active registration service for listening to and detecting devices registered with the platform, including starting and stopping listening.

3.1.2 Interface overview

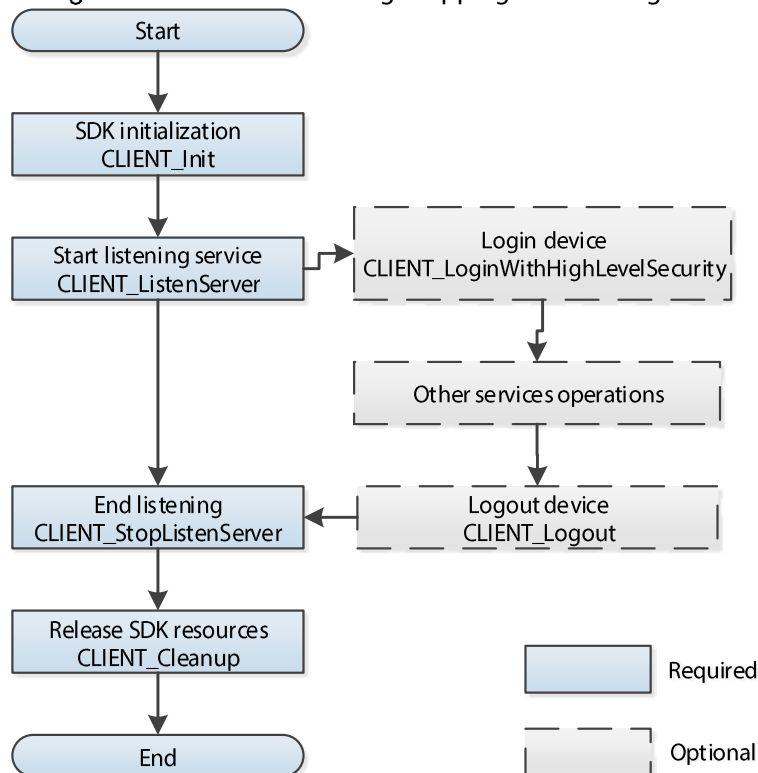
Table 3-1 Instructions for starting/stopping the active registration service interface

Interface	Description
CLIENT_Init	Initialize SDK
CLIENT_ListenServer	Start actively registering for listening
CLIENT_StopListenServer	Stop actively registering for listening
CLIENT_Cleanup	Clean up SDK resources

3.1.3 Process

The process for starting/stopping the platform active registration service is shown below.

Figure3-1 Process for starting/stopping the listening service



Process

- Step 1 Call the CLIENT_Init initialization interface to complete SDK initialization.
- Step 2 Call the CLIENT_ListenServer interface to start listening. The CLIENT_ListenServer interface requires the active registration listener callback function fServiceCallback to be passed in. Through this callback function, information such as the IP, ID, and port of the device registered to this service platform can be obtained. In the listening state, the callback function returns information such as the ID, IP, and port actively registered by the device. After obtaining the information in the callback, the user can log in to the required device

and perform other business operations.

Step 3 Call the CLIENT_StopListenServer interface to end the listening.

Step 4 Call the CLIENT_Cleanup interface to clean up and release SDK resources.

Precautions

- The server can only initiate communication with actively registered devices when it is in listening mode. Therefore, all operations that require the server to issue commands to the device must be completed in listening mode and cannot be interrupted midway.
- If you need to call SDK interfaces in the callback function, please start a separate thread to execute it; otherwise, it may cause the program to crash.

3.1.4 Example code

```
// For general code related to initialization, resource cleanup, etc., please refer to the "NetSDK Programming Guide".
///// Initialize active registration information
typedef struct tagDH_DEVICE_INFO
{
    char        szDevSerial[64];    /// Device ID
    char        szIP[64];          /// IP
    int         nPort;              /// Port
    char        szUserName[64];     /// Username
    char        szPassword[64];     /// Password
    LLONG       lLoginID;           /// Login handle
}DH_DEVICE_INFO;

DH_DEVICE_INFO m_DeviceInfo = {0};

///// Sets the listening IP for active registration, synchronously transmitting the login account, password, and
device ID from the actively registered device's web settings.
void GetLocalIPAndPort(char *szIP, int nLen, int *nPort)
{
    if(szIP == NULL || nLen<=0 || nPort == NULL)
        return;
    strncpy(szIP,m_szClientIP,nLen-1);
    *nPort = m_nPort;
    strncpy(m_DeviceInfo.szDevSerial,m_szDevSerial,sizeof(m_DeviceInfo.szDevSerial)-1);
    strncpy(m_DeviceInfo.szUserName,m_szUserName,sizeof(m_DeviceInfo.szUserName)-1);
    strncpy(m_DeviceInfo.szPassword,m_szPassword,sizeof(m_DeviceInfo.szPassword)-1);
}

///// Handle actively registered callback information
void DealListenCB(LLONG IHandle, char *pIp, WORD wPort, LONG ICommand, void *pParam, DWORD
dwParamLen)
{
    if (DH_DVR_SERIAL_RETURN == ICommand)
    {
        char* szDevSerial =(char*)pParam;

        if (strcmp(szDevSerial,m_DeviceInfo.szDevSerial) == 0)
```

```

        {
            strncpy(m_DeviceInfo.szIP,plp,sizeof(m_DeviceInfo.szIP)-1);
            m_DeviceInfo.nPort = wPort;
        }
    }
}

int CALLBACK ListenCallback(LLONG IHandle, char *plp, WORD wPort, LONG ICommand, void *pParam,
DWORD dwParamLen, LDWORD dwUserData)
{
    if(NULL != plp && NULL != pParam)
    {
        DealListenCB(IHandle, plp,wPort, ICommand, pParam, dwParamLen);
    }
    return 1;
}

void TestAutoReg()
{
    char szIP[32] = {0};
    int nPort = 0;
    GetLocalIPAndPort(szIP, sizeof(szIP),&nPort);
    LLONG m_IListenHdl = CLIENT_ListenServer(szIP, nPort, 1000, ListenCallback, NULL);
    if (0 == m_IListenHdl)
    {
        std::cout <<"Listen Fail! ip:"<<szIP<<"port:"<<nPort<< std::endl;
        return;
    }
    else
    {
        std::cout <<"Listen Success! ip:"<<szIP<<"port:"<<nPort<< std::endl;
        while(true)
        {
            if(strlen(m_DeviceInfo.szIP)!=0)
            {
                ///// If the IP address contains information, proceed with the login process.
                std::cout << "Device IP address detected: " << m_DeviceInfo.szIP << std::endl;
                std::cout << "Do you want to log in?" (y/n): " << std::endl;;
                char choice;
                std::cin >> choice;
                if (choice == 'y' || choice == 'Y')
                {
                    std::cout << "Attempting to log in..." << std::endl;
                    NET_IN_LOGIN_WITH_HIGHLEVEL_SECURITY stInparam;
                    memset(&stInparam, 0, sizeof(stInparam));
                    stInparam.dwSize = sizeof(stInparam);
                    strncpy(stInparam.szIP, m_DeviceInfo.szIP , sizeof(stInparam.szIP) - 1);
                    strncpy(stInparam.szPassword, m_DeviceInfo.szPassword,
sizeof(stInparam.szPassword) - 1);

```



```

        strncpy(stInparam.szUserName, m_DeviceInfo.szUserName,
sizeof(stInparam.szUserName) - 1);
        stInparam.nPort = m_DeviceInfo.nPort;
        stInparam.emSpecCap = EM_LOGIN_SPEC_CAP_SERVER_CONN;
        stInparam.pCapParam = m_DeviceInfo.szDevSerial;
        NET_OUT_LOGIN_WITH_HIGHLEVEL_SECURITY stOutparam;
        memset(&stOutparam, 0, sizeof(stOutparam));
        stOutparam.dwSize = sizeof(stOutparam);
        LLONG ILoginID = CLIENT_LoginWithHighLevelSecurity(&stInparam, &stOutparam);
        if (0 == ILoginID)
        {
            std::cout <<"CLIENT_LoginWithHighLevelSecurity
fail,error:"<<stOutparam.nError<< std::endl;
        }
        else
        {
            std::cout <<"CLIENT_LoginWithHighLevelSecurity
success,ILoginID:"<<ILoginID<< std::endl;
            m_DeviceInfo.ILoginID = ILoginID;
        }
        std::cout << "Exit operation?" ;
        std::cin >> choice;
        break;
    }
    else if (choice == 'n' || choice == 'N')
    {
        std::cout << "Cancel login operation" << std::endl;
        break;
    }
    else {
        std::cout << "Invalid input, please enter y or n" << std::endl;
    }
}

}
BOOL bRet = CLIENT_StopListenServer(m_IListenHdl);
if (TRUE != bRet)
{
    std::cout <<"Stop Listen Fail!"<< std::endl;
}
else
{
    std::cout <<"Stop listen Succeed!"<< std::endl;
}
m_IListenHdl = 0;
memset(&m_DeviceInfo,0,sizeof(m_DeviceInfo));
return;
}

```

}

Appendix 1 Security Recommendation

1. Account Management

1.1 Use Strong Passwords

- The length should not be less than 8 characters.
- Include at least two types of characters; character types include upper and lower case letters, numbers and symbols.
- Do not contain the account name or the account name in reverse order.
- Do not use continuous characters, such as 123, abc, etc.
- Do not use overlapped characters, such as 111, aaa, etc.

1.2 Change Password Regularly

It is suggested that you change passwords regularly to reduce the risk of being guessed or cracked.

1.3 Assign Accounts and Permissions Reasonably

According to business and management needs, reasonably add new users, and reasonably allocate a minimum set of permissions for them.

1.4 Enable Account Lock

The account lock feature is enabled by default, and it is recommended to keep it on to guarantee the account security. If an attacker attempts to log in with the wrong password several times, the corresponding account and the source IP address will be locked.

1.5 Set and Update Passwords Reset Information Timely

The platform supports password reset function. To reduce the risk of being attacked, please set up related information for password reset in time. If the information changes, please modify it in time. When setting password protection questions, it is suggested not to use those that can be easily guessed.

1.6 Enable Account Binding IP/MAC

It is recommended to enable the account binding IP/MAC mechanism to further improve access security.

2. Service Configuration

2.1 Enable HTTPS

It is suggested to enable HTTPS, so that you visit web service through a secure communication channel.

2.2 Disable Unnecessary Services and Choose Secure Modes

If not needed, it is recommended to turn off some services such as SNMP, SMTP, etc., to reduce risks.

If necessary, it is highly recommended that you use safe modes, including but not limited to the following services:

- SMTP: Choose TLS to access mailbox server.
- FTP: Choose SFTP, and set up strong passwords.

3. Network Configuration

3.1 Enable Firewall Allowlist

It is suggested to enable allowlist function to prevent everyone, except those with specified IP addresses, from accessing the system. Therefore, please be sure to add your computer's IP address

and the accompanying equipment's IP address to the allowlist.

3.2 Network Isolation

The network should be isolated by partitioning the video monitoring network and the office network on the switch and router to different VLANs. This prevents attackers from using the office network to launch Pivoting attacks on the video monitoring network.

4. Security Auditing

4.1 Check Online Users

It is recommended to check online users irregularly to identify whether there are illegal users logging in.

4.2 View the Platform Log

By viewing the log, you can get the IP information of the attempt to log in to the platform and the key operation information of the logged-in user.

5. Physical Protection

It is suggested to perform physical protection to the device that has installed the platform. For example, place the device in a special computer room and cabinet, and implement well-done access control permission and key management to prevent unauthorized personnel from carrying out physical contacts such as damaging hardware.

6. Perimeter Security

It is suggested to deploy perimeter security products and take necessary measures such as authorized access, access control, and intrusion prevention to protect the software platform security.